

NoodleAI Gesture Trainer

User Guide for ESP32-S3-Matrix

Collect • Train • Deploy • Infer

The screenshot shows the NoodleAI Gesture Trainer application window. At the top, it says "NoodleAI Gesture Trainer — ESP32-S3-Matrix". Below this is a "Device" section with a "Connect" button and a status "Disconnected". It also displays sensor data: ax: +0.142 g, ay: +0.372 g, az: +0.939 g, gx: +2.7 °/s, gy: -1.3 °/s, gz: +0.1 °/s. Below the device section are tabs: "Live IMU", "Dataset & Train", "Training Curves", and "Deploy & Infer". The "Dataset & Train" tab is active, showing three sections: 1. Dataset setup, 2. Record examples, and 3. Train N-layer MLP with scikit-learn. Section 1 includes a "Normalized gesture length" input (100) and a list of labels (1, 2, 3, 4). Section 2 includes a "Label" dropdown (4) and a "Use BOOT to record" button. Section 3 includes an "Input representation" dropdown (Accelerometer), "Hidden layers" input (64,32,16), "Epochs" input (300), and a "Train" button. The bottom of the window shows training progress: train 100.0% | validation 95.2% | export match 100.0%.

Desktop and browser workflows for NoodleAI NAI4 gesture recognition

Quick workflow

1 Connect	2 Set labels	3 Record	4 Train	5 Deploy	6 Infer
----------------------------	-------------------------------	---------------------------	--------------------------	---------------------------	--------------------------

1. What the GUI Does

NoodleAI Gesture Trainer provides one workflow for collecting motion gestures from the ESP32-S3-Matrix, training an MLP classifier, packaging the trained model, deploying it to the board, and testing inference. The GUI is organized into four tabs: **Live IMU**, **Dataset & Train**, **Training Curves**, and **Deploy & Infer**.

The board always streams raw six-axis IMU data (accelerometer + gyroscope). The selected input representation is chosen at training time, so the same recorded raw dataset can be reused for several feature representations.

1.1 Before starting

Item	What is needed
Hardware	ESP32-S3-Matrix with the NoodleAI firmware already flashed.
Bluetooth	Laptop/PC Bluetooth enabled.
Python GUI	Launch the NoodleAI Gesture Trainer application.
Gesture protocol	Press and hold BOOT to start a gesture; release BOOT to end it.

2. Connect to the ESP32-S3-Matrix

1. Power the ESP32-S3-Matrix and wait for the NoodleAI firmware to start.
2. Click Connect in the Device panel at the top of the GUI.
3. Wait until the status changes from Disconnected to Connected.
4. Move the board and confirm that ax, ay, az and gx, gy, gz update in the Device strip.

Tip: The Device strip is always visible, independent of the selected tab. This makes it easy to verify the BLE connection and live sensor values before recording or inference.

3. Live IMU Tab

Use the Live IMU tab to verify that the board is streaming meaningful motion data before collecting a dataset. The accelerometer channels are shown in g and the gyroscope channels in degrees per second.

- Keep the board stationary: one accelerometer axis should normally be near ± 1 g depending on orientation, while gyroscope values should stay near zero.
- Rotate or move the board: the traces should react immediately.
- If the plot is flat or no values update, fix the BLE/IMU connection before collecting samples.

4. Dataset & Train Tab

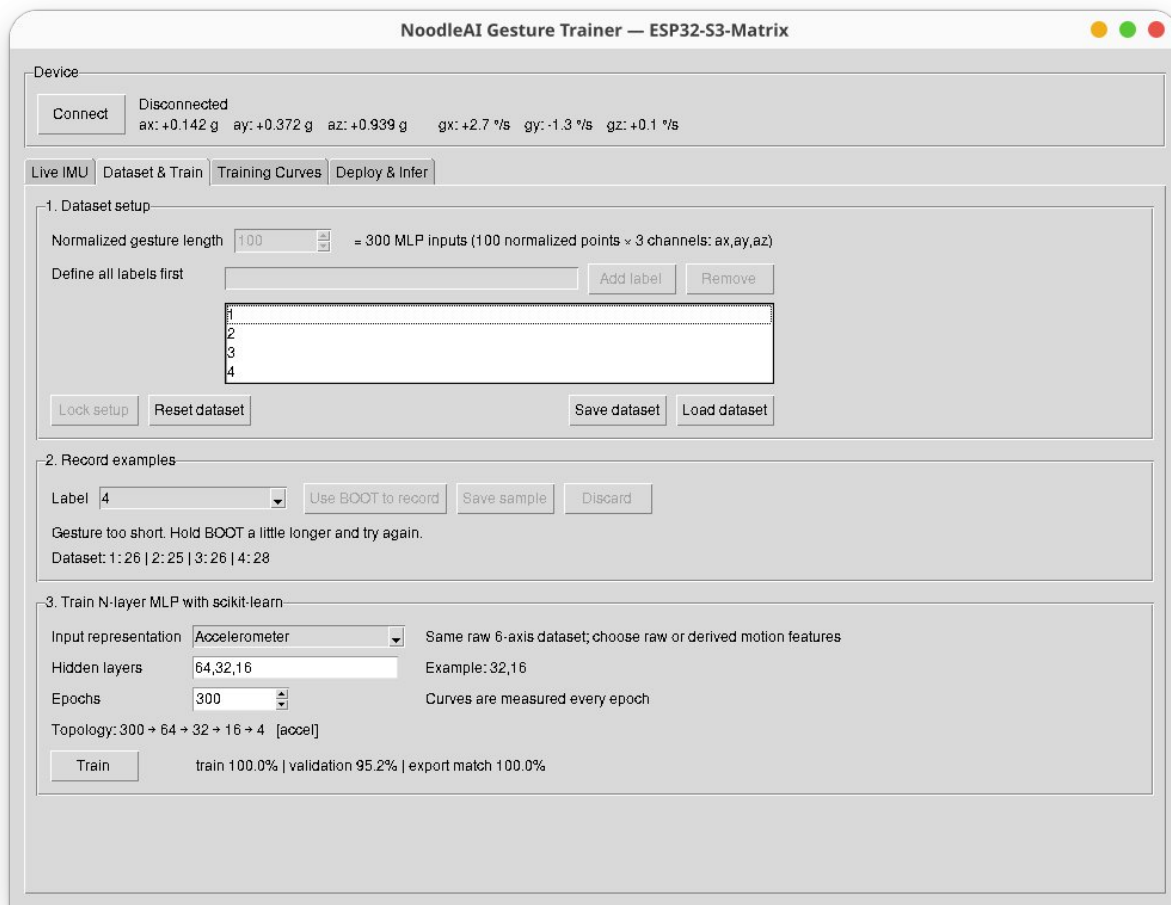


Figure 1. Dataset setup, recording controls, and MLP training settings.

4.1 Step 1 — Dataset setup

Normalized gesture length is the fixed number of points used to represent every gesture after temporal normalization. For example, with 100 normalized points and the Accelerometer representation (3 channels), the MLP input dimension is:

$$100 \text{ points} \times 3 \text{ channels} = 300 \text{ inputs}$$

1. Choose the normalized gesture length. The default value of 100 is a good starting point.
2. Enter each class label in Define all labels first and click Add label. For air-written digits, labels can simply be 1, 2, 3, 4, etc.
3. After all labels are defined, click Lock setup. The normalized length and label list are then fixed for the current dataset.

Define all labels before recording. Reset dataset unlocks the setup, but it also clears the current recorded dataset.

4.2 Step 2 — Record examples

Select the class to record from the Label drop-down, then use the BOOT button on the ESP32-S3-Matrix as the gesture delimiter.

1. Choose a label in the Label drop-down.
2. Click Use BOOT to record if the GUI is not already ready for recording.
3. Press and hold BOOT.
4. While holding BOOT, perform the complete gesture in free air.
5. Release BOOT when the gesture is finished.
6. Check the capture message. If the sample is correct, click Save sample; otherwise click Discard.
7. Repeat for every label. Try to collect a reasonably balanced number of samples per class.

Dataset counter. The line such as “Dataset: 1:26 | 2:25 | 3:26 | 4:28” shows the number of saved examples for each class.

Gesture too short. If the GUI reports that the gesture is too short, hold BOOT slightly longer and perform the gesture again. Very short captures are rejected before temporal normalization.

4.3 Save and load datasets

Use Save dataset to preserve the collected data and Load dataset to continue an earlier experiment. NAI4 datasets retain the raw six-axis gesture data, allowing different input representations to be trained later without recollecting the gestures.

4.4 Step 3 — Train the N-layer MLP

The training panel controls the representation, MLP topology, and number of training epochs.

Input representation	Channels / meaning	Typical input at N=100
Accelerometer	ax, ay, az	300
Gyroscope	gx, gy, gz	300
Accel + Gyro	ax, ay, az, gx, gy, gz	600
Relative Quaternion	qw, qx, qy, qz	400
Estimated Velocity	vx, vy, vz	300
Velocity + Quaternion	vx, vy, vz, qw, qx, qy, qz	700

Hidden layers. Enter comma-separated layer sizes. For example, “64,32,16” produces a topology such as 300 → 64 → 32 → 16 → 4 for a 4-class accelerometer problem.

Epochs. The default 300 epochs is suitable for the small educational MLPs used in NoodleAI. The training curves help determine whether more or fewer epochs are needed.

1. Choose the input representation.
2. Enter the hidden-layer sizes.
3. Set the number of epochs.
4. Check the displayed topology before training.
5. Click Train.
6. After training, review the reported training and validation accuracy and inspect the Training Curves tab.

5. Training Curves Tab

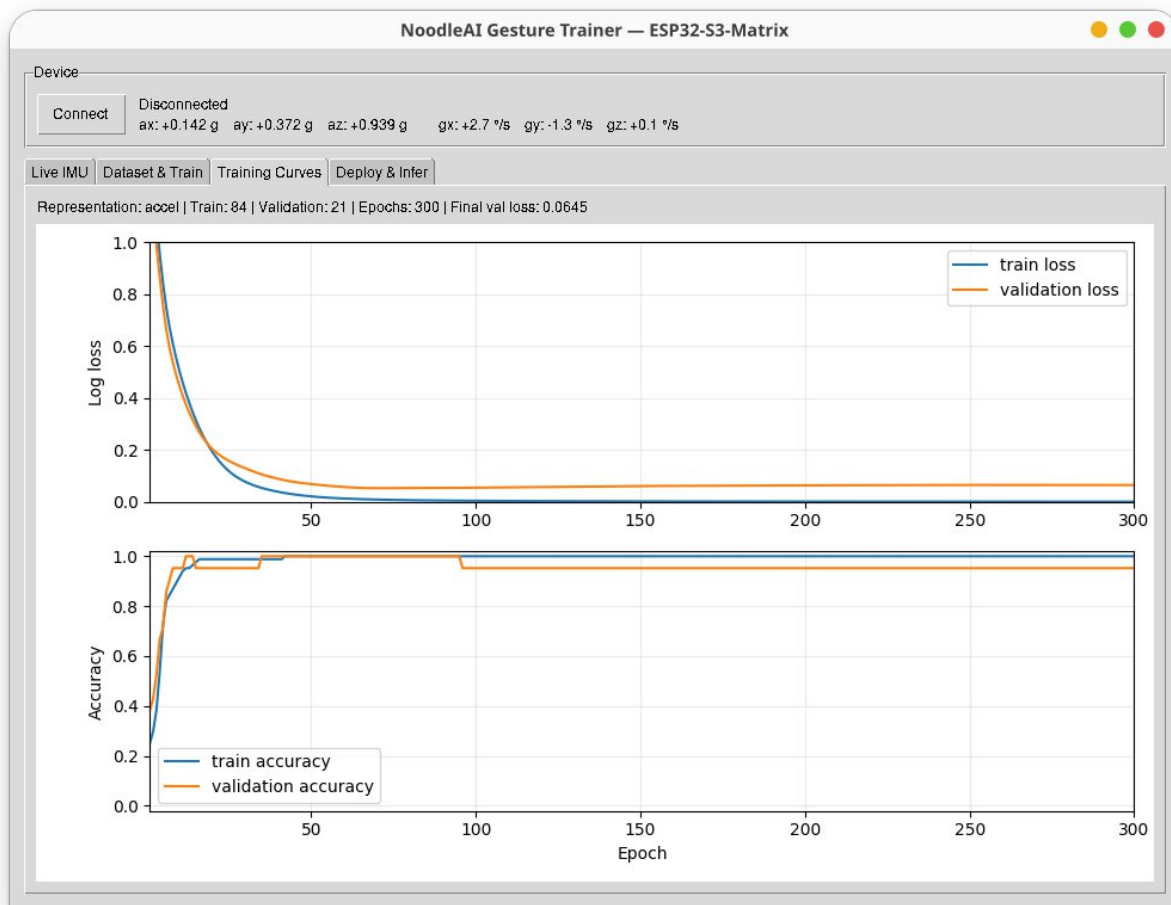


Figure 2. Training and validation loss/accuracy curves.

The upper plot shows training and validation log loss. The lower plot shows training and validation accuracy. These curves are more informative than the final accuracy alone.

- Healthy learning: training and validation loss both decrease and then stabilize.
- Possible overfitting: training loss keeps decreasing while validation loss starts to rise.
- Small datasets can produce deceptively high validation accuracy. Record more examples and test across different sessions or users before treating 100% as conclusive.
- If validation accuracy is much lower than training accuracy, collect more varied examples, simplify the network, or reconsider the representation.

6. Deploy & Infer Tab

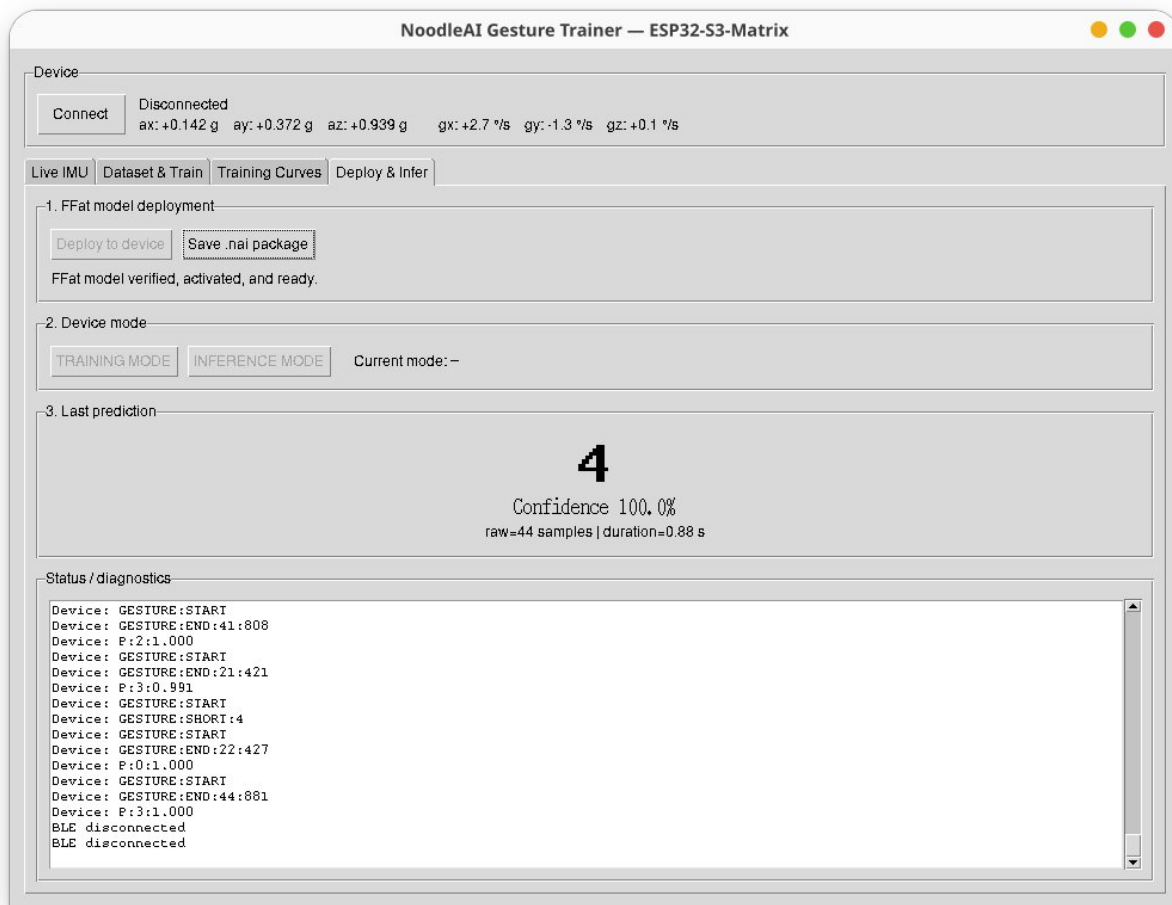


Figure 3. Model deployment, mode control, prediction display, and diagnostic log.

6.1 Deploy the trained model

1. Make sure the ESP32-S3-Matrix is connected.
2. After a successful training run, open Deploy & Infer.
3. Click Deploy to device.
4. Wait until the deployment status reports that the FFat model is verified, activated, and ready.

The model package contains the NAI4 configuration, StandardScaler parameters, and MLP weights/biases. The board stores the model in FFat and Noodle performs inference from the deployed parameters.

6.2 Save a model package

Click Save .nai package to keep the trained model as a portable NoodleAI package. This is useful for archiving experiments or redeploying the same model later.

6.3 Switch device mode

Use TRAINING MODE when collecting labeled gestures and INFERENCE MODE when testing a deployed classifier. The Current mode field shows the mode reported by the board.

6.4 Run inference

1. Switch the board to INFERENCE MODE.

2. Press and hold BOOT.
3. Perform the gesture.
4. Release BOOT.
5. Read the predicted label and confidence in Last prediction.
6. Use Status / diagnostics to inspect the gesture length, device messages, and prediction packet.

Example. In Figure 3, the predicted class is “4” with 100.0% confidence. The diagnostic log also records GESTURE:START, GESTURE:END, and prediction messages such as P:2:1.000.

7. Recommended Experimental Practice

- Keep class counts balanced. A large imbalance can bias the classifier.
- Record natural variation: slightly different speed, size, starting angle, and trajectory.
- Use the same physical gesture convention across classes and sessions.
- Do not judge the model only from training accuracy; check validation curves and perform fresh gestures after deployment.
- For comparisons among Accelerometer, Gyroscope, Quaternion, or Velocity representations, reuse the same raw NAI4 dataset so the comparison is fair.
- Save both the dataset and the .nai package for experiments that may be reported later.

8. Troubleshooting

Symptom	What to check
Cannot connect	Confirm Bluetooth is enabled, the board is powered, and no other application is already holding the BLE connection.
Sensor values do not change	Check the live IMU stream before collecting data. Reconnect if necessary.
Gesture too short	Hold BOOT longer and draw the complete gesture before releasing it.
Train button disabled	Lock the dataset setup and make sure at least two labels are defined.
Poor validation accuracy	Collect more varied samples, keep classes balanced, or try another input representation/topology.
Deploy button disabled	A model must first be trained successfully and the board must be connected.
No prediction	Verify that a valid model is deployed and the device is in INFERENCE MODE.

9. One-Page Checklist

- ☐ Connect the ESP32-S3-Matrix and confirm live IMU values.
- ☐ Set normalized gesture length.
- ☐ Define every class label, then Lock setup.

- ☐ Choose a label, hold BOOT, perform gesture, release BOOT.
- ☐ Save or discard the captured sample; repeat for all classes.
- ☐ Save the dataset when enough samples have been collected.
- ☐ Choose input representation, hidden layers, and epochs.
- ☐ Train and inspect validation accuracy + training curves.
- ☐ Deploy to device and wait for model verification.
- ☐ Switch to INFERENCE MODE and test fresh gestures.
- ☐ Save the .nai model package for reproducibility.

Core idea: record raw motion once, choose the representation at training time, then deploy only the model.
This keeps NoodleAI flexible for both teaching and experiments.

10. NoodleAI Web Version

NoodleAI Web provides the same collect–train–deploy–infer workflow as the desktop GUI, but it runs inside a browser. Raw six-axis IMU samples are received through Web Bluetooth, the MLP is trained locally with TensorFlow.js, the NAI4 package is created in the browser, and the model is deployed back to the ESP32-S3-Matrix over Bluetooth.

Public web app: <https://auralius.github.io/nai/>

Web quick workflow

1 Open	2 Pair	3 Collect	4 Train	5 Deploy	6 Infer
------------------	------------------	---------------------	-------------------	--------------------	-------------------

10.1 Open the web app and pair the device

Open the NoodleAI Web address in a Chromium-based browser with Web Bluetooth enabled. Click Connect, then select the NoodleAI device in the browser pairing dialog. The browser asks for permission before the page can access the Bluetooth device.

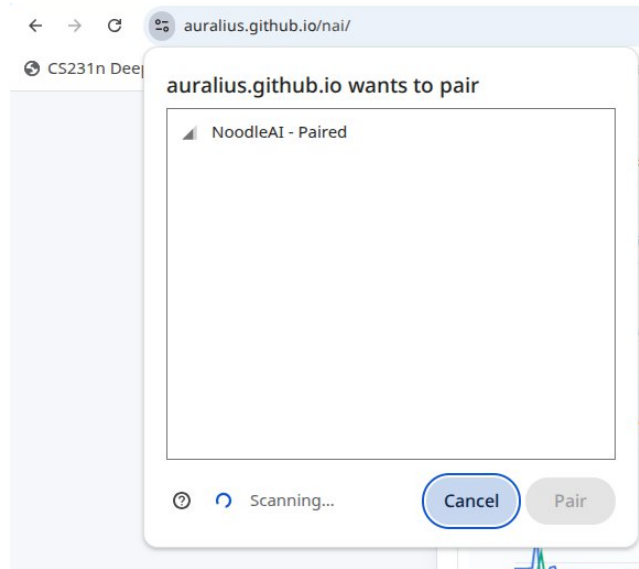


Figure 4. Browser Web Bluetooth pairing dialog for NoodleAI.

- If the device is already known to the computer, it may appear as “NoodleAI - Paired”.
- After pairing, the page should show Connected and the accelerometer/gyroscope values should update.
- The web app is client-side: gesture data, preprocessing, and TensorFlow.js training run locally in the browser.

10.2 Live IMU tab

The Live IMU tab is the browser equivalent of the desktop Live IMU view. It displays the raw QMI8658 accelerometer and gyroscope stream received over Web Bluetooth. Use this page to verify the connection before recording a dataset.



Figure 5. NoodleAI Web Live IMU tab showing raw accelerometer and gyroscope traces.

- A stationary board should show approximately 1 g along the gravity direction and small gyroscope rates.
- Move or rotate the board and verify that both plots respond immediately.
- Click Clear plot when a clean live view is needed.

10.3 Dataset & Train tab

The browser follows the same three-stage workflow as the desktop GUI: dataset setup, record examples, and train the N-layer MLP. The raw six-axis gesture is retained so the same dataset can be reused for different NAI4 representations.

NoodleAI Web
Collect. Train. Upload. Infer.

TF.js 4.22.0 - cpu Connected

Disconnect Connected: NoodleAI
Raw six-axis stream active.

Accel
ax 0.186 g ay 0.030 g az 0.983 g

Gyro
gx 7.2 °/s gy 3.2 °/s gz 1.2 °/s

Live IMU Dataset & Train Training Curves Deploy & Infer

1 Dataset setup

Define the normalized gesture length and all class labels before recording.

Normalized gesture length: 100

Stored raw stream: 6 IMU channels

Current input: $100 \times 3 = 300$

Class label, e.g. 1

Add label

1
2
3
4

Remove selected

Lock setup Reset dataset

Load .npz Save .npz

2 Record examples

Choose a label, hold BOOT while drawing, then release BOOT.

Label: 1

Use BOOT to record Save sample Discard

Loaded 105 samples from angka1to3.npz.

Total samples: 105

Classes: 4

Raw NAI4 gestures available

1: 26 2: 25 3: 26 4: 28

3 Train N-layer MLP

Training runs locally in this browser with TensorFlow.js.

Input representation: Accelerometer

Hidden layers: 64,32,16

Epochs: 50

Topology: 300 → 64 → 32 → 16 → 4 [Accelerometer]

Train Train 98.8% · validation 100.0% · NAI4 88.1 KiB

Figure 6. NoodleAI Web Dataset & Train tab.

Dataset setup.

- Set the normalized gesture length and define all labels before recording.
- Click Lock setup after the label list is complete.
- Use Load .npz and Save .npz to exchange datasets with the desktop NoodleAI application.

Record examples.

- Select a label, click Use BOOT to record, hold BOOT while performing the gesture, then release BOOT.
- Review the captured gesture and choose Save sample or Discard.
- Keep the class counts reasonably balanced and record natural variation in speed, size, and trajectory.

Train N-layer MLP.

- Choose the input representation: Accelerometer, Gyroscope, Accel + Gyro, Relative Quaternion, Estimated Velocity, or Velocity + Quaternion.
- Enter the hidden layers as comma-separated sizes, for example 64,32,16.
- Set the epoch count and check the displayed topology before clicking Train.
- TensorFlow.js trains the model locally in the browser; the resulting model is exported to the same binary NAI4 format used by the device.

10.4 Training Curves tab

The Training Curves tab shows loss and accuracy for the training and validation subsets. Interpret these plots in the same way as the desktop version: falling loss and stable validation accuracy indicate useful learning, while a widening train/validation gap can indicate overfitting.

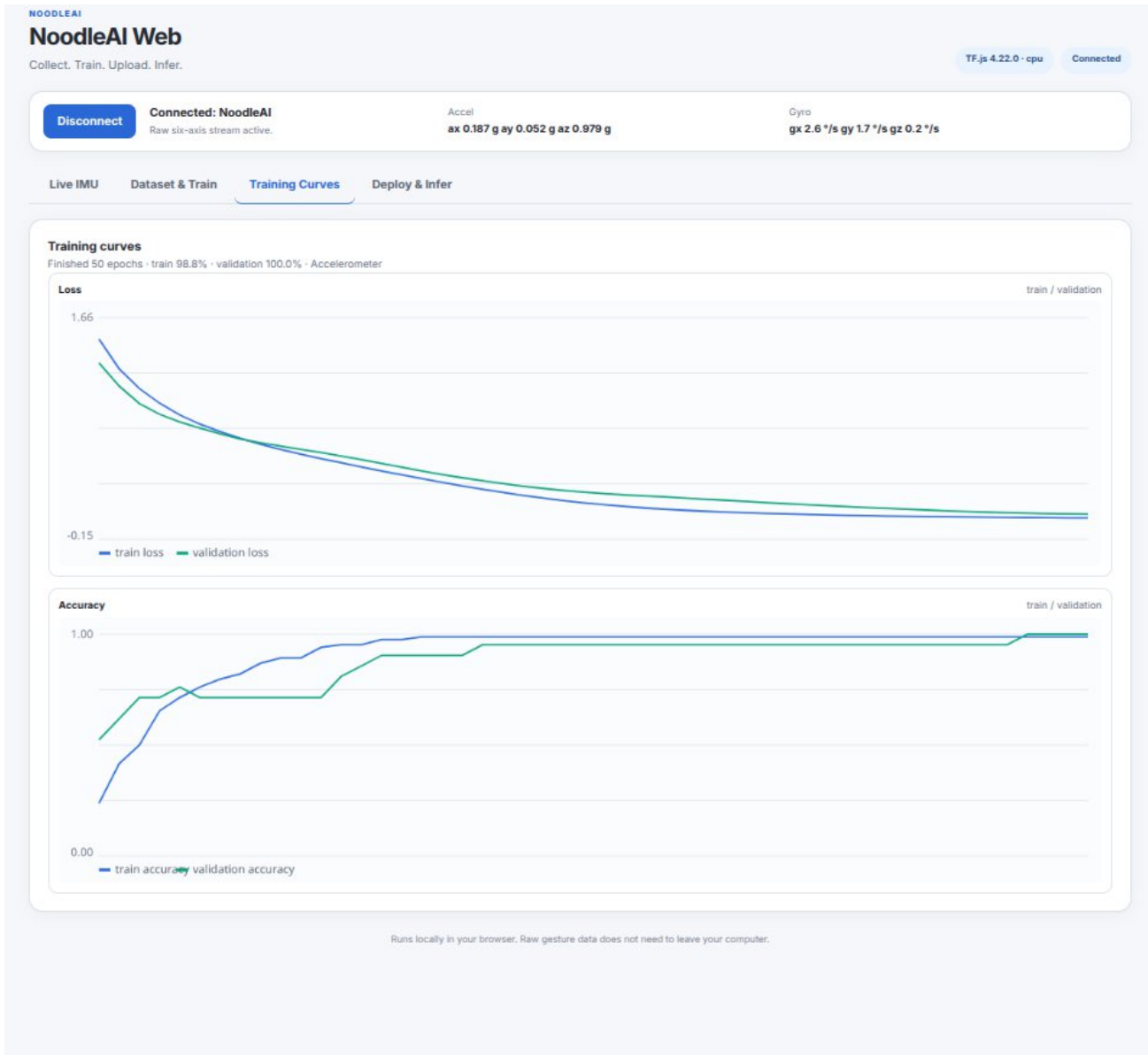


Figure 7. TensorFlow.js training and validation curves in NoodleAI Web.

10.5 Deploy & Infer tab

After training, the browser creates the NAI4 package in memory. The package can be saved locally as a .nai file or transmitted directly to the ESP32-S3-Matrix through Web Bluetooth. Deployment uses the same file-level verification and activation workflow as the desktop application.

NoodleAI Web
Collect. Train. Upload. Infer.

TF.js 4.22.0 - cpu Connected

Disconnect Inference: 2 (100.0%)
Raw six-axis stream active.

Accel
ax -0.723 g ay 0.188 g az 0.652 g

Gyro
gx 4.0 °/s gy -14.6 °/s gz 2.7 °/s

Live IMU Dataset & Train Training Curves **Deploy & Infer**

1 FFat model deployment
Upload the current binary NAI4 package without recompiling firmware.

Deploy to device **Save .nai package**

BLE chunk: 160 bytes

No model deployed in this session.

2 Device mode
Training streams raw IMU data; inference executes the deployed representation and Noodle MLP.

TRAINING MODE **INFERENCE MODE** Current mode: INFERENCE

3 Last prediction

2
Confidence 100.0%
raw=43 samples · duration=0.85 s

Status / diagnostics **Clear**

```
[12:02:38 AM] TensorFlow.js 4.22.0 ready (cpu).
[12:02:55 AM] BLE connected to NoodleAI.
[12:02:55 AM] Device: CONNECTED:M
[12:04:06 AM] BLE disconnected.
[12:05:09 AM] BLE ERROR: User cancelled the requestDevice() chooser.
[12:05:18 AM] BLE connected to NoodleAI.
[12:05:18 AM] Device: CONNECTED:M
[12:06:23 AM] Loaded angkalt03.npz: 105 samples; raw gestures available.
[12:07:07 AM] Training complete: Accelerometer, topology 300-64-32-16-4, train=100.0%, validation=100.0%, NAI4=88.1 KiB.
[12:07:21 AM] Training complete: Accelerometer, topology 300-64-32-16-4, train=98.8%, validation=100.0%, NAI4=88.1 KiB.
[12:07:44 AM] Device: MODE:I
[12:07:46 AM] Device: GESTURE:START
[12:07:47 AM] Device: GESTURE:END:23:470
[12:07:47 AM] Device: P:0:1.000
[12:07:48 AM] Device: GESTURE:START
```

Figure 8. NoodleAI Web deployment, mode control, prediction result, and diagnostic log.

1. Make sure the board is connected and a model has been trained.
2. Click Deploy to device and wait until the status/diagnostic log reports successful model activation.
3. Use TRAINING MODE for data collection or INFERENCE MODE for prediction.
4. In inference mode, hold BOOT, perform the gesture, and release BOOT.
5. Read the predicted label and confidence in Last prediction. The diagnostic log also shows gesture and prediction messages.
6. Use Save .nai package when a portable copy of the trained model is needed.

10.6 Desktop and Web versions at a glance

Function	Desktop GUI	NoodleAI Web
BLE communication	Python / Bleak	Web Bluetooth
Training engine	scikit-learn MLPClassifier	TensorFlow.js
Dataset	NAI4 .npz	NAI4 .npz (desktop-compatible)
Model package	Binary .nai / NAI4	Binary .nai / NAI4
Deployment	BLE to FFat	Web Bluetooth to FFat
On-device inference	Noodle runtime	Noodle runtime

Important: The desktop and web trainers may learn different numerical weights because their training implementations and random initialization details differ. Both export the same NAI4 deployment format, and both are executed by the same Noodle inference runtime on the ESP32-S3-Matrix.

10.7 Web quick checklist

- ☐ Open <https://auralius.github.io/nai/> and connect NoodleAI through the browser pairing dialog.
- ☐ Confirm the Live IMU plots respond to board motion.
- ☐ Define labels and normalized gesture length, then Lock setup.
- ☐ Record BOOT-delimited gestures and save each accepted sample.
- ☐ Choose representation, hidden layers, and epochs; then Train.
- ☐ Inspect Training Curves and validation accuracy.
- ☐ Deploy the generated NAI4 model over Web Bluetooth.
- ☐ Switch to INFERENCE MODE and test fresh gestures.
- ☐ Save .npz and .nai files when the experiment should be reproduced later.